

**Projectnaam:** TaskManagerMaeconomy

**Framework:** React Native (Cross-platform) & Firebase Cloud Services

**Projectbeheer:** Trello (<https://trello.com/b/QUYER3Qf/to-do-list-react>)

## 1. DOELSTELLING

Dit Functioneel Ontwerp beschrijft de werking, de gebruikersinteractie en de logische structuur van de applicatie **TaskManagerMaeconomy**. De applicatie is ontwikkeld tijdens de stageperiode als kennismaking met het React Native framework.

Het hoofddoel is om een soepel werkende applicatie te bieden waarmee gebruikers hun dagelijkse taken kunnen beheren, inclusief functionaliteiten voor authenticatie en profielpersonalisatie.

## 2. GEBRUIKERSROLLEN

Het systeem kent twee typen eindgebruikers (actoren) met elk hun eigen rechten binnen de applicatie:

1. **Anonieme Bezoeker (Gast):** Heeft uitsluitend toegang tot het authenticatieproces. Kan de app opstarten, een nieuw account registreren en inloggen.
2. **Geregistreerde Gebruiker:** Heeft na succesvolle authenticatie toegang tot de afgeschermdede kernfunctionaliteiten (het dashboard, de taakbeheer-CRUD-acties en de profielinstellingen).

## 3. INTERFACE-ONTWERP

De applicatie is opgebouwd uit drie hoofdschermen om een overzichtelijke gebruikerservaring (UX) te garanderen.

### 3.1 Authenticatiescherm

- **Invoer:** Tekstvelden voor E-mailadres en Wachtwoord.
- **Acties:** Knoppen voor "Inloggen" en "Registreren".
- **Validatie:** Visuele meldingen bij lege velden of een onjuist e-mailformaat.

### 3.2 Hoofddashboard

- **Bovenkant:** Header met app-titel en een snelkoppeling naar het profielscherm.
- **Midden (Lijstweergave):** Een soepel scrollbare lijst (`FlatList`) met actuele taken. Elk item toont de titel en de huidige status (Open/Voltooid).
- **Onderkant:** Een invoerveld met een actieknop (+) om direct een taak toe te voegen.

### 3.3 Profielscherm

- **Weergave:** Huidige profielfoto (avatar) en het e-mailadres van de ingelogde gebruiker.
- **Acties:** Knop om een nieuwe foto uit de galerij te selecteren en te uploaden, en een knop om veilig uit te loggen.

## 4. FUNCTIONELE SPECIFICATIES

De functionele behoeften zijn geprioriteerd via de MoSCoW-methode om de scope van het project te bewaken.

### 4.1 Must Haves

- **US01 - Account Registratie:** Anonieme bezoekers moeten een account kunnen aanmaken via Firebase.
- **US02 - Inlogsysteem:** Veilige toegang verlenen aan geregistreerde gebruikers; onjuiste inlogpogingen tonen een veilige foutmelding.
- **US03 - Real-time Databasekoppeling:** Transacties worden direct gesynchroniseerd met de cloud-database. Data-isolatie is verplicht: gebruikers kunnen uitsluitend hun eigen taken inzien op basis van hun unieke User ID (UID).
- **US04 - Profielfoto Uploaden:** Gebruikers kunnen via Firebase Storage een eigen afbeelding uploaden en koppelen aan hun profiel.

### 4.2 Should Haves

- **US05 - Taken Toevoegen:** Gebruikers kunnen via het dashboard nieuwe taken aan de database toevoegen, mits de invoer niet leeg is.
- **US06 - Taken Bewerken:** Het wijzigen van de tekst of status (bijv. vinkje zetten bij voltooid) wordt direct verwerkt in de UI en database.
- **US07 - Taken Verwijderen:** Taken kunnen permanent worden gewist via een prullenbak-icoon of swipe-gebaar.

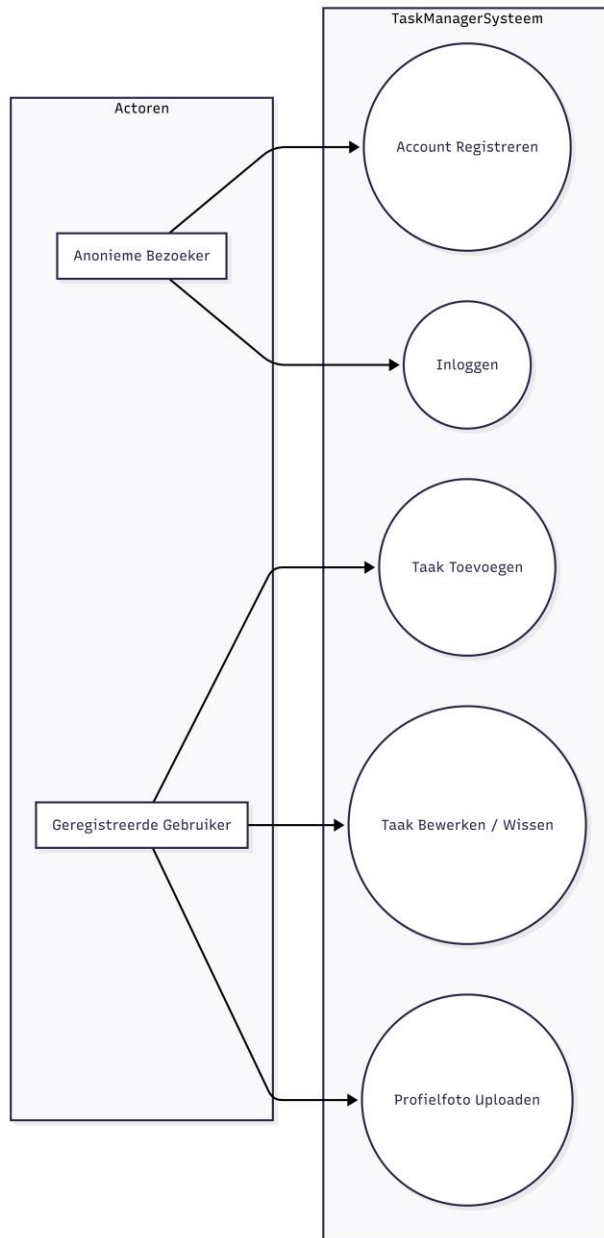
### 4.3 Could Haves

- **US08 - Sorteren:** Mogelijkheid om taken lokaal (in de frontend state) alfabetisch of op aanmaakdatum te sorteren.
- **US09 - Filteren:** De lijst dynamisch filteren op tabbladen (bijv. "Alle", "Openstaand", "Voltooid") zonder dat er data in de database wijzigt.

## 5. ONTWERPSCHEMA'S & ARCHITECTUUR

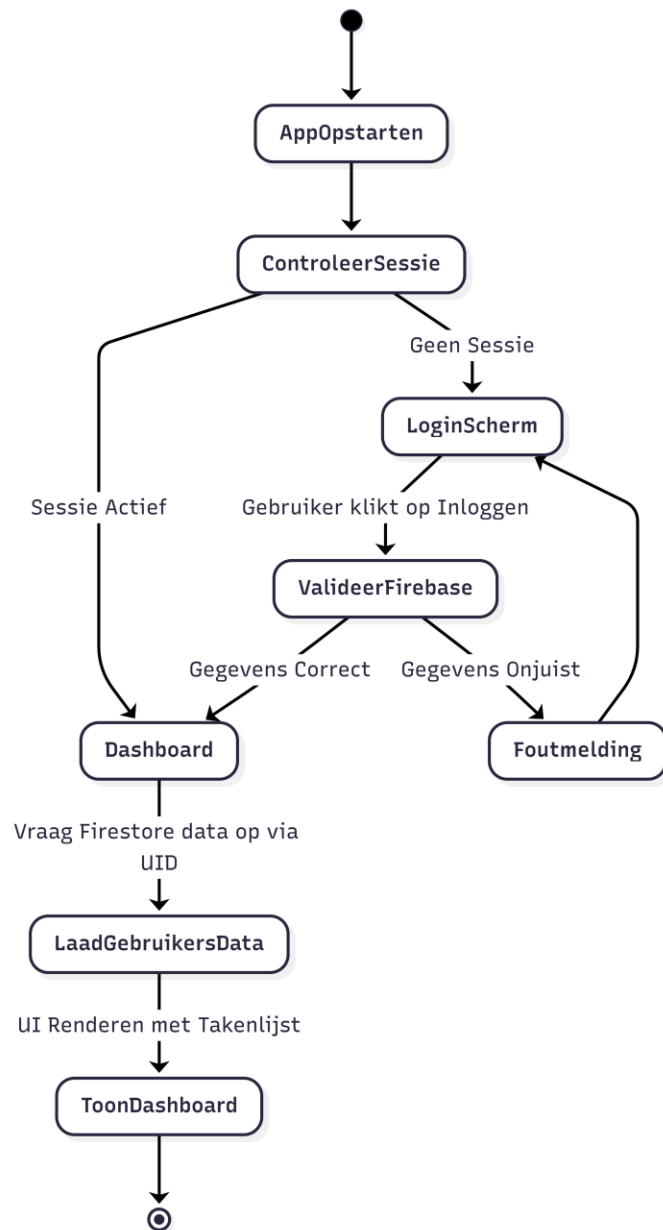
### 5.1 Use-Case Diagram

Dit diagram toont de scope van de applicatie en welke actor welke handelingen mag verrichten.



### 5.2 Activiteitendiagram (Workflow Inloggen & Taken Laden)

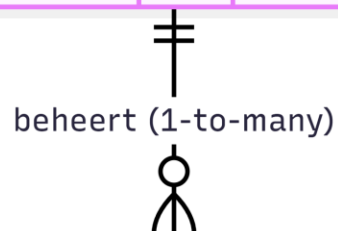
Dit schema toont de logische stappen die het systeem doorloopt bij de start en validatie van de gebruiker.



### 5.3 Entity Relationship Diagram (ERD)

Dit model toont hoe de gegevens logisch gestructureerd zijn en hoe de 1-op-veel (1-to-many) relatie is opgebouwd.

| USER   |                     |    |                            |
|--------|---------------------|----|----------------------------|
| string | uid                 | PK | Unique Firebase ID         |
| string | email               |    | Gebruikers e-mail          |
| string | profile_picture_url |    | Link naar Firebase Storage |
| date   | created_at          |    |                            |



| TODO_ITEM |              |    |                         |
|-----------|--------------|----|-------------------------|
| string    | id           | PK | Unique Task ID          |
| string    | user_id      | FK | Koppeling naar USER.uid |
| string    | title        |    | Inhoud van de taak      |
| boolean   | is_completed |    | Status van de taak      |
| date      | timestamp    |    |                         |

## 6. NON-FUNCTIONELE EISEN

- **Codeconventies:** Consistent gebruik van camelCasing voor variabelen/functies en PascalCasing voor React Native schermen/componenten (zoals LoginScreen.js en TodoItem.js).
- **Beveiliging & Privacy (AVG):** Er worden geen wachtwoorden in plain-text opgeslagen; authenticatie verloopt via de encryptie-protocollen van Firebase. Er is sprake van strikte data-isolatie (User-scoping), zodat gebruikers elkaars persoonsgegevens en taken nooit kunnen inzien.

- **Voortgangscntrole:** De realisatie van deze functionele onderdelen is bijgehouden en overzien middels een Trello-Scrumboard.